

Enterprise Workforce AI Platform Technical Documentation

- [Enterprise Workforce AI Platform — Technical Documentation](#)
 - [Table of Contents](#)
 - [1. Executive Summary](#)
 - [2. System Architecture](#)
 - [3. Technology Stack](#)
 - [4. Project Structure](#)
 - [5. Database Schema & Data Models](#)
 - [6. Backend API Reference](#)
 - [7. Frontend Architecture](#)
 - [8. Authentication & Security](#)
 - [9. Role-Based Access Control \(RBAC\)](#)
 - [10. Module Implementation Status](#)
 - [11. Key Features & Capabilities](#)
 - [12. Configuration & Environment](#)
 - [13. Development Setup](#)
 - [14. Deployment Notes](#)
 - [15. Known Issues & Technical Debt](#)

Enterprise Workforce AI Platform — Technical Documentation

Version: 1.0 | **Date:** July 2026 | **Status:** Active Development **Repository:** <https://github.com/Aditya-Khatri-7/enterprise-workforce-ai-platform> **Team:** Aditya, Saurabh, Dhumi, Madhur, Siddhant

Table of Contents

- [1. Executive Summary](#)
 - [2. System Architecture](#)
 - [3. Technology Stack](#)
 - [4. Project Structure](#)
 - [5. Database Schema & Data Models](#)
 - [6. Backend API Reference](#)
 - [7. Frontend Architecture](#)
 - [8. Authentication & Security](#)
 - [9. Role-Based Access Control \(RBAC\)](#)
 - [10. Module Implementation Status](#)
 - [11. Key Features & Capabilities](#)
 - [12. Configuration & Environment](#)
 - [13. Development Setup](#)
 - [14. Deployment Notes](#)
 - [15. Known Issues & Technical Debt](#)
-

1. Executive Summary

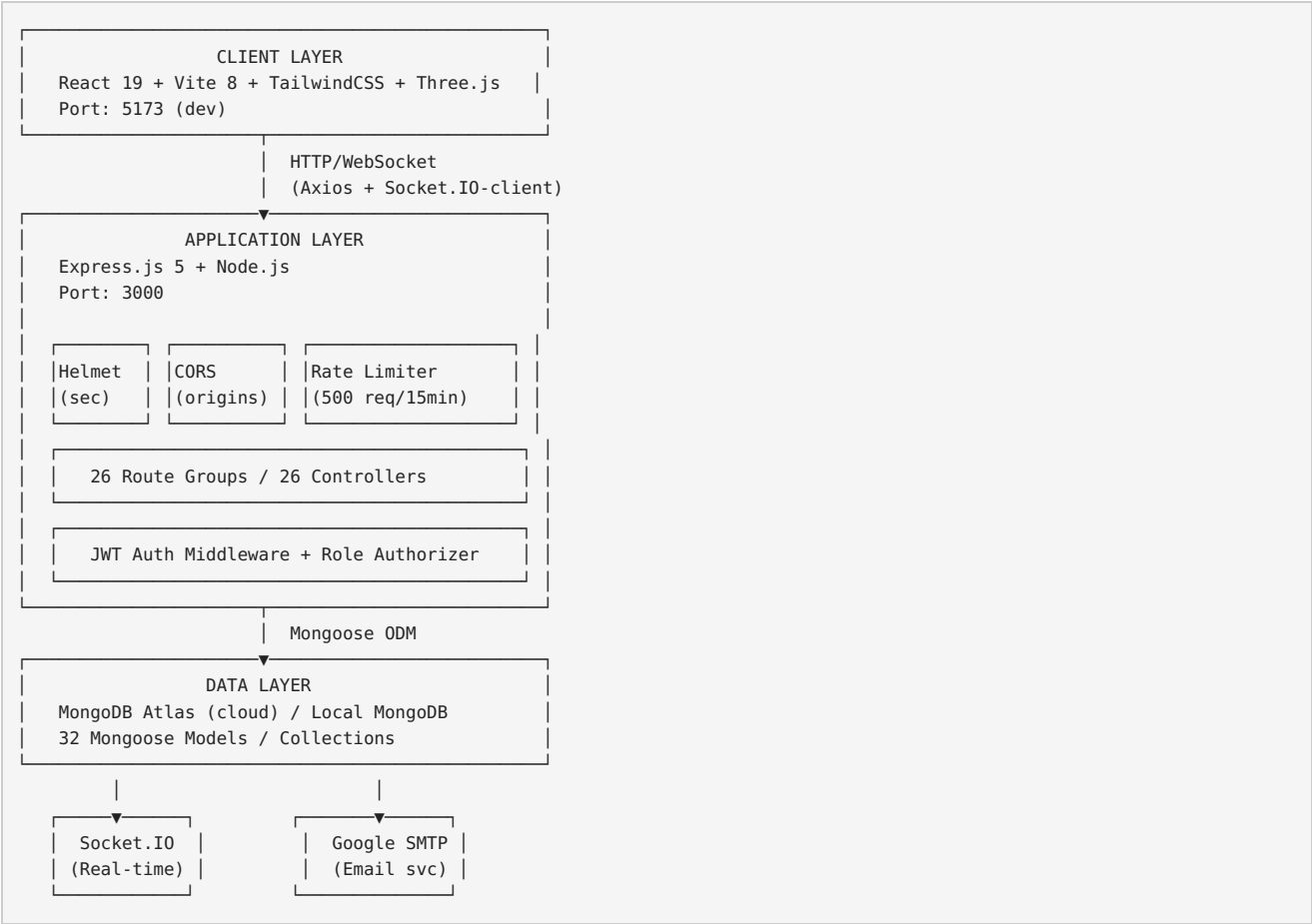
The **Enterprise Workforce AI Platform (EWAP)** is a full-stack MERN (MongoDB, Express.js, React, Node.js) web application designed to digitize and automate the complete employee lifecycle for medium to large organizations. It provides a centralized hub for managing employees, departments, attendance, leave, payroll, projects, assets, and

support tickets — all enhanced by an embedded AI Operations Assistant.

Key Differentiators

- **Multi-tenant architecture** — one platform instance supports multiple independent organizations
- **Granular RBAC** — 9 distinct roles with per-route authorization
- **Dual-mode operation** — Live database mode AND a fully-offline Demo Mode powered by a client-side mock data engine
- **Real-time capabilities** — Socket.IO integration for live notifications
- **AI-powered assistant** — Integrated AI chat widget accessible across all pages
- **Premium UI** — Glassmorphism design with Framer Motion animations, 3D Three.js backgrounds, and a Guided Tour showcase mode

2. System Architecture



Communication Pattern

- **Frontend → Backend:** RESTful HTTP via Axios with HTTP-only cookie authentication
- **Auth Strategy:** Access Token (short-lived JWT in HTTP-only cookie) + Refresh Token rotation
- **Real-time:** Socket.IO for notifications and live updates
- **Demo Mode:** All API calls are intercepted by a client-side Axios adapter that serves data from `mockDataEngine.js`

3. Technology Stack

Backend

--	--	--

Technology	Version	Purpose
Node.js	LTS	Runtime environment
Express.js	^5.2.1	Web application framework
Mongoose	^9.7.3	MongoDB ODM
jsonwebtoken	^9.0.3	JWT generation & verification
bcrypt	^6.0.0	Password hashing
Socket.IO	^4.8.3	Real-time WebSocket communication
Nodemailer	^9.0.3	Email service via Google SMTP
Helmet	^8.2.0	HTTP security headers
CORS	^2.8.6	Cross-Origin Resource Sharing
express-rate-limit	^8.5.2	API rate limiting
cookie-parser	^1.4.7	HTTP cookie parsing
dotenv	^17.4.2	Environment variable management
nodemon	^3.1.14	Dev auto-reload

Frontend

Technology	Version	Purpose
React	^19.2.7	UI component library
Vite	^8.1.1	Build tool & dev server
React Router DOM	^7.18.1	Client-side routing
Axios	^1.18.1	HTTP client
Framer Motion	^12.42.2	Animations & transitions
Three.js	^0.185.1	3D background graphics
@react-three/fiber	^9.6.1	React renderer for Three.js
@react-three/drei	^10.7.7	Three.js helpers
Recharts	^3.9.2	Data visualization charts
Lucide React	^1.23.0	Icon library
Socket.IO Client	^4.8.3	Real-time communication
React Toastify	^11.1.0	Toast notifications
React Hook Form	^7.80.0	Form state management
TailwindCSS	^3.4.19	Utility-first CSS framework

Database

Technology	Purpose
MongoDB Atlas	Cloud-hosted production database
Local MongoDB	Local development fallback

4. Project Structure

```
enterprise-workforce-ai-platform/
├── backend/
│   ├── controllers/
│   │   ├── aiController.js      # AI assistant query handler
│   │   ├── assetController.js   # Company asset tracking
│   │   ├── attendanceController.js # Attendance clock-in/out
│   │   └── auditController.js   # Audit log retrieval
│   └── # Node.js/Express API server
│       # Business logic (26 files)
```

- └─ authController.js # Login, logout, token refresh, OTP
- └─ departmentController.js # Department CRUD
- └─ designationController.js # Designation CRUD
- └─ employeeController.js # Employee lifecycle management
- └─ enterpriseController.js # Enterprise-level data (payroll, leaves)
- └─ grievanceController.js # Employee grievance handling
- └─ leaveController.js # Leave request workflow
- └─ notificationController.js # In-app notifications
- └─ objectionController.js # Work objection management
- └─ officeLocationController.js # Office branch management
- └─ organizationController.js # Org CRUD & admin provisioning
- └─ payrollController.js # Payroll entry management
- └─ policyController.js # Company policy management
- └─ progressReportController.js # Project progress reports
- └─ projectController.js # Project assignment & workflow
- └─ proposalController.js # Project proposals
- └─ recruitmentController.js # Candidate & job posting management
- └─ requestController.js # General approval requests
- └─ supportController.js # Help desk tickets
- └─ teamController.js # Team management
- └─ userController.js # User account management
- └─ workShiftController.js # Work shift CRUD
- └─ models/ # Mongoose schemas (32 files)
 - └─ Asset.js # Company asset schema
 - └─ Attendance.js # Daily attendance record
 - └─ AttendanceCorrection.js # Correction requests
 - └─ AuditLog.js # System audit trail
 - └─ Candidate.js # Recruitment candidate
 - └─ Department.js # Department structure
 - └─ Designation.js # Job designation/role title
 - └─ Employee.js # Core employee profile
 - └─ Grievance.js # Employee grievance record
 - └─ JobPosting.js # Open positions
 - └─ LeaveRequest.js # Leave application
 - └─ Notification.js # In-app notifications
 - └─ OfficeLocation.js # Office branch locations
 - └─ Organization.js # Tenant organization
 - └─ Otp.js # OTP for password reset
 - └─ Payroll.js # Salary record
 - └─ PerformanceReview.js # Employee performance review
 - └─ Permission.js # Granular permissions
 - └─ Policy.js # Company policies
 - └─ ProgressReport.js # Project progress report
 - └─ Project.js # Project with team assignment
 - └─ ProjectProposal.js # Proposed projects
 - └─ ProjectRequest.js # Project change requests
 - └─ RefreshToken.js # Refresh token store
 - └─ Request.js # General approval requests
 - └─ Role.js # User role definitions
 - └─ SupportRequest.js # Help desk ticket
 - └─ Task.js # Project task
 - └─ TeamRequest.js # Team lead change requests
 - └─ User.js # User authentication record
 - └─ WorkObjection.js # Work objection records
 - └─ WorkShift.js # Work shift configuration
- └─ middlewares/
 - └─ auth.js # JWT verification + role authorization
- └─ routes/ # API route definitions (26 files)
 - └─ ai.js # POST /api/ai/chat
 - └─ asset.js # CRUD /api/assets
 - └─ attendance.js # /api/attendance
 - └─ audit.js # GET /api/audit
 - └─ auth.js # /api/auth (login, logout, refresh)
 - └─ department.js # CRUD /api/departments
 - └─ designation.js # CRUD /api/designations
 - └─ employee.js # CRUD /api/employees
 - └─ enterprise.js # /api/enterprise
 - └─ grievance.js # /api/grievances
 - └─ leave.js # /api/leaves
 - └─ notification.js # /api/notifications

```

├── objection.js           # /api/objections
├── officeLocation.js      # CRUD /api/office-locations
├── organization.js        # CRUD /api/organizations
├── payroll.js             # /api/payroll
├── policy.js              # /api/policies
├── progressReport.js      # /api/progress-reports
├── project.js             # /api/projects
├── proposal.js            # /api/proposals
├── recruitment.js         # /api/recruitment
├── request.js             # /api/requests
├── support.js             # /api/support
├── team.js                # /api/teams
├── user.js                # /api/users
├── workShift.js           # /api/workshifts

├── utils/
│   ├── emailService.js    # Nodemailer Google SMTP integration
│   ├── jwt.js             # JWT sign/verify helpers
│   ├── notification.js     # Notification creation helper
│   └── socket.js          # Socket.IO server initialization

├── server.js              # Express app entry point
├── seed.js                # Database seeder script
├── clear_db.js            # DB reset utility
├── list_users.js          # Dev utility: list all users
├── reset_superadmin.js    # Super admin credential reset
├── package.json

├── frontend/              # React/Vite application
│   ├── src/
│   │   ├── App.jsx        # Root component with router config
│   │   ├── main.jsx        # React DOM entry point
│   │   └── index.css       # Global styles
│   │
│   │   ├── context/
│   │   │   ├── AuthContext.jsx # Global auth state (user, login, logout)
│   │   │   ├── DemoContext.jsx # Demo mode state & role switcher
│   │   │   └── ThemeContext.jsx # Light/Dark theme toggle
│   │
│   │   ├── services/
│   │   │   └── api.js       # Axios instance with interceptors & demo adapter
│   │
│   │   ├── routes/
│   │   │   ├── ProtectedRoute.jsx # Guards all authenticated routes
│   │   │   └── RoleRoute.jsx      # Guards routes by allowed role list
│   │
│   │   ├── layouts/
│   │   │   └── DashboardLayout.jsx # Sidebar + header shell for all dashboards
│   │
│   │   ├── hooks/          # Custom React hooks
│   │
│   │   ├── utils/
│   │   │   ├── mockDataEngine.js # Client-side mock API for Demo Mode (19KB)
│   │   │   └── soundManager.js    # UI sound effects manager
│   │
│   │   ├── components/
│   │   │   ├── AIChatWidget.jsx  # Floating AI assistant chat (7.7KB)
│   │   │   ├── CustomCursor.jsx  # Custom animated cursor (3.4KB)
│   │   │   ├── PremiumBackground.jsx # Animated Three.js background (9.7KB)
│   │   │   ├── ai/               # AI-specific sub-components
│   │   │   ├── boot/             # App boot/loading screens
│   │   │   └── showcase/
│   │   │       ├── AIExplainer.jsx # Showcase AI explanation overlay
│   │   │       ├── DemoBadge.jsx   # Demo mode badge + Command Center (17KB)
│   │   │       └── GuidedTour.jsx  # Automated role walkthrough tour (19KB)
│   │
│   │   └── pages/              # Route-level page components (28 files)
│   │       ├── LandingPage.jsx    # Public homepage (40.5KB)
│   │       ├── Login.jsx          # Authentication page (14KB)
│   │       ├── ForgotPassword.jsx  # OTP-based password reset (8.6KB)
│   │       ├── AdminDashboard.jsx  # Master dashboard for all admin roles (272KB)
│   │       ├── EmployeeDashboard.jsx # Employee self-service portal (38.4KB)
│   │       └── EmployeeManagement.jsx # HR employee management (20.2KB)

```

├─ ManagerTeamLeads.jsx	# Dept. manager → team lead oversight (11.5KB)
├─ ManagerProjectRequests.jsx	# Project change request approvals (6.5KB)
├─ ManagerTeamRequests.jsx	# Team lead vote requests (7.9KB)
├─ ManagerGrievances.jsx	# Grievance portal for managers (7.3KB)
├─ TeamLeadMyTeam.jsx	# Team lead → team & project view (6.4KB)
├─ TeamLeadRequests.jsx	# Team lead approvals (7.7KB)
├─ TeamLeadRatings.jsx	# Employee rating dashboard (9.1KB)
├─ EmployeeProjects.jsx	# Employee project acceptance (7.6KB)
├─ EmployeeHistory.jsx	# Project & task history log (3.5KB)
├─ EmployeeCompanyInfo.jsx	# Resume edit & ratings view (9.2KB)
├─ EmployeeLeaves.jsx	# Leave application portal (10.3KB)
├─ EmployeeTickets.jsx	# Help desk ticket portal (9.5KB)
├─ EmployeeTeamLeadRequests.jsx	# Team lead change voting (11KB)
├─ EmployeeProjectRequests.jsx	# Project change requests (11.3KB)
├─ SuperAdminRequestCenter.jsx	# Super admin approval center (16.5KB)
├─ AuditLogPage.jsx	# Audit trail viewer (7.4KB)
├─ Profile.jsx	# User profile editor (17.9KB)
├─ AccountStatus.jsx	# Inactive/suspended account screen (6KB)
├─ AIWelcomePage.jsx	# AI onboarding welcome screen (4.2KB)
├─ DemoPlaceholderPage.jsx	# Demo mode landing (14.1KB)
├─ CreateOrgPlaceholderPage.jsx	# New org creation flow (3.5KB)
├─ DeactivatedPage.jsx	# Suspended account page (7.7KB)
├─ index.html	# HTML entry point
├─ vite.config.js	# Vite bundler configuration
├─ tailwind.config.js	# TailwindCSS configuration
├─ package.json	
├─ public/	# Legacy/static assets
├─ index.html	
├─ app.js	
├─ style.css	
├─ README.md	# Project documentation (28KB)
├─ planning.md	# Development roadmap
├─ converted.md	# Requirements specification (24KB)
├─ .gitignore	
├─ package-lock.json	

5. Database Schema & Data Models

Core Models

User (Authentication Record)

```

{
  "email": "String (unique, required)",
  "username": "String (unique, required)",
  "password": "String (bcrypt hashed)",
  "role": "ObjectId → Role",
  "employeeRef": "ObjectId → Employee",
  "organization": "ObjectId → Organization",
  "mustChangePassword": "Boolean (default: true)",
  "failedLoginAttempts": "Number (default: 0)",
  "isLocked": "Boolean",
  "lockUntil": "Date",
  "isActive": "Boolean (default: true)",
  "status": "Enum: Active | Inactive | Suspended | Locked | Pending | Archived | Deactivation_Requested | Deleted",
  "suspendedBy": "ObjectId → User",
  "suspendReason": "String",
  "reactivationRequest": {
    "requestedAt": "Date",
    "reason": "String",
    "status": "Enum: Pending | Approved | Rejected"
  }
}
```

Employee (Core Profile)

```

{
  "employeeId": "String (unique, auto-generated)",
  "firstName": "String (required)",
  "lastName": "String (required)",
  "gender": "Enum: Male | Female | Other",
  "dob": "Date",
  "mobile": "String",
  "email": "String (unique, required)",
  "address": "String",
  "bloodGroup": "String",
  "department": "String (required)",
  "designation": "String (required)",
  "reportingManager": "ObjectId → Employee",
  "joiningDate": "Date (required)",
  "employmentType": "Enum: Full-Time | Part-Time | Contract",
  "status": "Enum: Active | On Leave | Terminated | Archived",
  "profileImage": "String (URL/base64)",
  "emergencyContact": "String",
  "skills": ["String"],
  "userRef": "ObjectId → User",
  "resumeText": "String (inline editable)",
  "resumeFileName": "String",
  "resumeFileBase64": "String",
  "ratings": {
    "teamLeadRating": "Number (default: 0)",
    "managerRating": "Number (default: 0)"
  }
}

```

Organization (Multi-Tenant Root)

```

{
  "organizationId": "String (unique, required)",
  "name": "String (unique, required)",
  "email": "String (unique, required)",
  "address": "String",
  "subscriptionPlan": "Enum: Basic | Premium | Enterprise",
  "status": "Enum: Active | Suspended | Deleted"
}

```

Project (Project Assignment)

```

{
  "name": "String (required)",
  "description": "String",
  "organization": "ObjectId → Organization",
  "department": "String",
  "assignedTeamLead": "ObjectId → Employee",
  "employees": ["ObjectId → Employee"],
  "status": "Enum: Ongoing | Completed",
  "pendingAgreement": "Boolean",
  "agreedEmployees": ["ObjectId → Employee"],
  "tlAcceptedStatus": "Enum: Pending | Accepted | Rejected",
  "rejectedEmployees": ["ObjectId → Employee"]
}

```

Payroll (Salary Record)

```

{
  "employee": "ObjectId → Employee",
  "baseSalary": "Number (required)",
  "allowances": "Number (default: 0)",
  "deductions": "Number (default: 0)",
  "netSalary": "Number (required)",
  "month": "String (e.g. 'January')",
  "year": "Number",
  "status": "Enum: Pending | Approved | Processed",
  "organization": "ObjectId → Organization",
}

```

```

    "salaryHold": {
      "isOnHold": "Boolean (default: false)",
      "heldBy": "ObjectId → User",
      "heldAt": "Date",
      "holdReason": "String",
      "releasedAt": "Date"
    }
  }
}

```

Grievance

```

{
  "employee": "ObjectId → Employee (required)",
  "organization": "ObjectId → Organization (required)",
  "title": "String (required)",
  "description": "String (required)",
  "status": "Enum: Pending | Resolved",
  "resolution": "String",
  "resolvedBy": "ObjectId → Employee"
}

```

Additional Models Summary

Model	Key Fields
Department	name, code, organization, status
Designation	name, code, organization
Attendance	employee, date, clockIn, clockOut, status
LeaveRequest	employee, type, startDate, endDate, status, approvedBy
SupportRequest	employee, title, description, type, status, assignedTo
Asset	name, type, assignedTo, serialNumber, status
AuditLog	action, performedBy, targetResource, timestamp
Notification	recipient, message, type, read, createdAt
Candidate	name, email, skills, experience, status, jobPosting
TeamRequest	fromTeam, proposedLead, votes, status, organization
Grievance	employee, title, description, status, resolution
ProgressReport	project, submittedBy, content, completionPercentage
WorkObjection	employee, project, reason, status
Request	type, requestedBy, status, details
OfficeLocation	name, address, totalEmployees, organization
WorkShift	name, startTime, endTime, organization
Policy	title, content, organization, isActive
Role	name, description
RefreshToken	token, user, expiresAt
Otp	email, otp, expiresAt
Task	title, project, assignedTo, status, dueDate
PerformanceReview	employee, reviewer, period, scores, comments

6. Backend API Reference

Server Configuration

- **Base URL:** `http://localhost:3000/api`

- **Rate Limiting:** 500 requests per 15 minutes (disabled in development)
- **Auth:** HTTP-only cookie (`accessToken`) + Refresh Token rotation

API Route Groups

Authentication — `/api/auth`

Method	Endpoint	Access	Description
POST	<code>/login</code>	Public	Authenticate user, issue JWT cookies
POST	<code>/logout</code>	Authenticated	Clear auth cookies
POST	<code>/refresh</code>	Public	Rotate access token using refresh token
GET	<code>/me</code>	Authenticated	Get current user profile
POST	<code>/forgot-password</code>	Public	Send OTP to email
POST	<code>/reset-password</code>	Public	Verify OTP and reset password
POST	<code>/change-password</code>	Authenticated	Change own password

Organizations — `/api/organizations`

Method	Endpoint	Access	Description
GET	<code>/</code>	Super Admin	List all organizations
POST	<code>/</code>	Super Admin	Create organization
PUT	<code>/:id</code>	Super Admin, Org Admin	Update organization
DELETE	<code>/:id</code>	Super Admin	Delete organization
POST	<code>/:id/admin</code>	Super Admin	Assign Organization Admin

Employees — `/api/employees`

Method	Endpoint	Access	Description
GET	<code>/</code>	HR, Admin	List all employees in org
POST	<code>/</code>	HR, Admin	Create employee record
GET	<code>/:id</code>	Authenticated	Get employee details
PUT	<code>/:id</code>	HR, Admin	Update employee details
DELETE	<code>/:id</code>	Admin	Archive/delete employee
PUT	<code>/:id/resume</code>	Employee (self)	Update resume text/file
PUT	<code>/:id/rate</code>	Team Lead, Manager	Submit employee rating
PUT	<code>/:id/suspend</code>	Admin, HR	Suspend employee
PUT	<code>/:id/activate</code>	Admin, HR	Reactivate employee

Projects — `/api/projects`

Method	Endpoint	Access	Description
GET	<code>/</code>	Authenticated	List projects for organization
POST	<code>/</code>	Manager, Dept Manager	Create project
PUT	<code>/:id</code>	Manager	Update project
DELETE	<code>/:id</code>	Admin, Manager	Delete project
PUT	<code>/:id/assign</code>	Manager	Assign team lead + employees
PUT	<code>/:id/tl-accept</code>	Team Lead	Accept project assignment
PUT	<code>/:id/reject</code>	Employee	Reject project assignment
PUT	<code>/:id/agree</code>	Employee	Agree to project assignment

Leave — /api/leaves

Method	Endpoint	Access	Description
GET	/	HR, Admin	List all leave requests
POST	/	Employee	Submit leave request
PUT	/:id/approve	HR, Manager	Approve leave
PUT	/:id/reject	HR, Manager	Reject leave
GET	/my	Employee	Get own leave history

Grievances — /api/grievances

Method	Endpoint	Access	Description
POST	/	Employee	Submit grievance
GET	/	Manager, Admin	View all grievances in org
PUT	/:id/resolve	Manager, Admin	Resolve grievance

Payroll — /api/payroll & /api/enterprise/payrolls

Method	Endpoint	Access	Description
GET	/	Finance, Admin	List payroll records
POST	/	Finance	Create payroll entry
PUT	/:id/hold	Finance	Place salary hold
PUT	/:id/release	Finance	Release salary hold

Support — /api/support

Method	Endpoint	Access	Description
GET	/	IT Admin, Admin	View all tickets
POST	/	Employee	Create support ticket
PUT	/:id	IT Admin	Update ticket status

Other Routes

Route Group	Key Endpoints
/api/attendance	clock-in, clock-out, corrections, monthly reports
/api/departments	CRUD, org-scoped
/api/designations	CRUD, org-scoped
/api/workshifts	CRUD, org-scoped
/api/office-locations	CRUD, org-scoped
/api/recruitment	candidates, job postings, status updates
/api/assets	asset CRUD, assign to employee
/api/teams	team management
/api/progress-reports	project progress reporting
/api/requests	general approval request management
/api/audit	audit log retrieval (Super Admin only)
/api/notifications	notification CRUD
/api/ai	AI assistant query proxy
/api/users	user account management
/api/policies	company policy CRUD

/api/proposals	project proposal management
/api/objections	work objection management

7. Frontend Architecture

Routing Strategy

The application uses React Router DOM v7 with a nested route structure:

```

/ (Public)
├─ /login
├─ /forgot-password
├─ /welcome
├─ /demo
├─ /create-organization
├─ /account-suspended
└─ [ProtectedRoute] (requires authentication)
    ├─ /dashboard → redirects by role
    └─ /account-status
        └─ [DashboardLayout] (sidebar + header)
            ├─ [RoleRoute: Admin roles]
            │   ├─ /admin/dashboard → AdminDashboard.jsx
            │   └─ /admin/requests → SuperAdminRequestCenter.jsx
            └─ [RoleRoute: HR + Admin]
                ├─ /admin/employees → EmployeeManagement.jsx
                └─ /admin/audit → AuditLogPage.jsx
                    └─ [RoleRoute: Dept Manager + Admin]
                        ├─ /admin/team-leads → ManagerTeamLeads.jsx
                        ├─ /admin/project-requests → ManagerProjectRequests.jsx
                        ├─ /admin/team-requests → ManagerTeamRequests.jsx
                        └─ /admin/grievances → ManagerGrievances.jsx
                            └─ [RoleRoute: Team Lead + Manager]
                                ├─ /admin/my-team → TeamLeadMyTeam.jsx
                                ├─ /admin/team-lead-requests → TeamLeadRequests.jsx
                                └─ /admin/employee-ratings → TeamLeadRatings.jsx
                                    └─ [RoleRoute: Employee]
                                        ├─ /employee/dashboard → EmployeeDashboard.jsx
                                        ├─ /employee/projects → EmployeeProjects.jsx
                                        ├─ /employee/history → EmployeeHistory.jsx
                                        ├─ /employee/company-info → EmployeeCompanyInfo.jsx
                                        ├─ /employee/leaves → EmployeeLeaves.jsx
                                        ├─ /employee/tickets → EmployeeTickets.jsx
                                        ├─ /employee/team-lead-requests → EmployeeTeamLeadRequests.jsx
                                        └─ /employee/project-requests → EmployeeProjectRequests.jsx
                                            └─ /profile → Profile.jsx

```

State Management

Context	Purpose
AuthContext	Global user session state (user object, login/logout functions, loading)
DemoContext	Demo mode flag, selected demo role, role switching
ThemeContext	Light/Dark theme preference

API Layer (services/api.js)

- Axios instance pointing to `http://{hostname}:3000/api`

- **Request Interceptor:** Checks `localStorage.ewap_demo_mode` — if `true`, routes all requests to `mockDataEngine.js`
- **Response Interceptor:** Handles `401 TOKEN_EXPIRED` with automatic refresh token rotation; redirects to `/account-suspended` on `ACCOUNT_SUSPENDED` error code

Demo Mode Architecture

The `mockDataEngine.js` (19KB) is a complete client-side mock backend that: - Intercepts all Axios requests in demo mode - Returns realistic organizational data for all 9 user roles - Simulates all major workflows without needing the backend - Enables showcase presentations without a live database

8. Authentication & Security

Authentication Flow

1. User submits credentials → `POST /api/auth/login`
2. Server validates email/password (`bcrypt compare`)
3. On success: generate short-lived Access Token (JWT) + long-lived Refresh Token
4. Both tokens set as HTTP-only cookies (not accessible via JavaScript)
5. Client stores no tokens in `localStorage`
6. On `401` response: client automatically calls `POST /api/auth/refresh`
7. Server validates Refresh Token, issues new Access Token
8. Original request retried automatically

Security Features

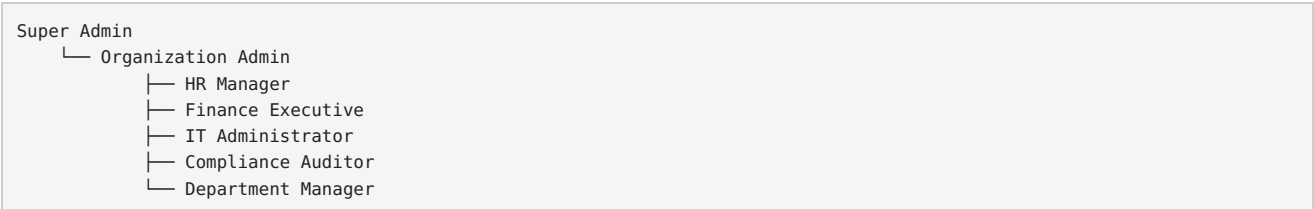
Feature	Implementation
Password hashing	<code>bcrypt</code> (cost factor 10+)
JWT expiry	Short-lived access tokens
Refresh token rotation	New refresh token on each use
HTTP-only cookies	Prevents XSS token theft
Account logout	After 5 failed login attempts
Rate limiting	500 req/15min per IP (disabled in dev)
CORS	Whitelisted origins only (<code>localhost:5173/5174/5175</code>)
Helmet	Standard HTTP security headers
Suspended account guard	Middleware blocks all API access except <code>auth/me</code> and reactivation
Role authorization	Middleware enforces role allowlists per route

Middleware Chain

Request → `Helmet` → `CORS` → `Rate Limiter` → `authenticateUser` → `authorizeRoles` → Controller

9. Role-Based Access Control (RBAC)

User Roles & Hierarchy



Team Lead

Employee

Role Capabilities Matrix

Capability	Super Admin	Org Admin	HR Manager	Dept Manager	Team Lead	Employee	Finance	IT Admin	Auditor
Create Organizations	✔	✗	✗	✗	✗	✗	✗	✗	✗
Manage All Employees	✔	✔	✔	✗	✗	✗	✗	✗	👁
Suspend/Deactivate Users	✔	✔	✔	✗	✗	✗	✗	✗	✗
Dept & Shift Config	✔	✔	✗	✗	✗	✗	✗	✗	✗
Approve Leaves	✔	✔	✔	✔	✗	✗	✗	✗	✗
Manage Projects	✔	✔	✗	✔	✔	✗	✗	✗	✗
Rate Employees	✔	✗	✗	✔	✔	✗	✗	✗	✗
Submit Grievances	✗	✗	✗	✗	✗	✔	✗	✗	✗
Resolve Grievances	✔	✔	✔	✔	✗	✗	✗	✗	✗
Payroll Management	✔	✗	✗	✗	✗	✗	✔	✗	✗
Asset Management	✔	✗	✗	✗	✗	✗	✗	✔	✗
Help Desk Management	✔	✗	✗	✗	✗	✗	✗	✔	✗
View Audit Logs	✔	✔	✗	✗	✗	✗	✗	✗	✔
Recruitment	✔	✔	✔	✗	✗	✗	✗	✗	✗

Note: Super Admin always bypasses all role checks (hardcoded in `authorizeRoles` middleware).

10. Module Implementation Status

Module	ID	Status	Completeness
Authentication & User Management	M-01	✔ Complete	~95%
Organization Management	M-02	✔ Complete	~90%
Employee Management	M-03	✔ Complete	~90%
Recruitment Management	M-04	🕸 Partial	~50% (backend exists, frontend limited)
Attendance Management	M-05	🕸 Partial	~60% (backend exists)
Leave Management	M-06	✔ Complete	~90%
Payroll Management	M-07	🕸 Partial	~65% (entry + hold system done)
Performance Management	M-08	🕸 Partial	~40% (ratings schema + TL/Mgr rating UI)
Project & Task Management	M-09	🕸 Partial	~70% (assignment + acceptance workflow done)
Asset Management	M-10	🕸 Partial	~50% (backend + IT admin UI)
Help Desk	M-11	✔ Complete	~85%
Document Management	M-12	🕸 Partial	~40% (resume upload/edit done)

Notifications	M-13	● Partial	~50% (email service + in-app schema)
Reports & Analytics	M-14	● Partial	~45% (dashboard KPIs, no export)
AI Operations Assistant	M-15	● Partial	~60% (chat widget + backend route)

Overall Estimated Completion: ~70%

11. Key Features & Capabilities

Multi-Role Dashboard System

The `AdminDashboard.jsx` (272KB) serves as the universal dashboard shell for all non-employee roles and renders different views based on the authenticated user's role: - Super Admin Panel: Global org management, activation requests - Organization Admin Panel: Dept/Shift/Location config, employee directory - HR Manager Panel: Employee lifecycle, leave approvals, recruitment - Finance Executive Panel: Payroll registry, salary holds, expense tracking - IT Administrator Panel: Asset management, help desk ticket resolution - Department Manager Panel: Team leads, projects, grievances - Team Lead Panel: My team, employee ratings - Auditor Panel: Read-only audit log access

Demo Showcase Mode

- Activated via the Demo badge on the landing page
- All API calls intercepted — no backend required
- 9 role selector allows instant persona switching
- **Guided Tour** (`GuidedTour.jsx`): Automated, scripted walkthrough with auto-navigation, role switching, Play/Pause/Skip controls
- **Command Center** (`DemoBadge.jsx`): Futuristic telemetry HUD overlay showing system vitals

AI Operations Assistant

- Floating chat widget accessible from all authenticated pages
- Backend route proxies queries to external AI API
- Contextually aware of user role

Grievance Management Workflow

1. Employee submits grievance via Employee Dashboard
2. Department Manager views pending grievances via `/admin/grievances`
3. Manager resolves with a resolution note
4. Status updates to `Resolved`

Team Lead Change (Democratic Voting)

1. Team members vote to change team lead (requires 2/3 majority)
2. Vote tracked in `TeamRequest` model
3. Department Manager reviews via `/admin/team-requests`
4. Manager approves, system updates team assignment

Project Assignment Workflow

1. Department Manager creates project, assigns Team Lead + Employees
2. Team Lead accepts/rejects via `/admin/employee-ratings` (TL accept)
3. Each employee accepts/rejects individually via `/employee/projects`
4. Department Manager monitors via project dashboard

12. Configuration & Environment

Backend .env Variables

```
# Required
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/ewap
JWT_SECRET=<your-secret-key>
JWT_REFRESH_SECRET=<your-refresh-secret-key>
PORT=3000
NODE_ENV=development

# Email Service
EMAIL_USER=<gmail-address>
EMAIL_PASS=<gmail-app-password>

# Optional
GEMINI_API_KEY=<google-gemini-api-key>
```

Frontend Environment

- No .env needed for development
- API base URL auto-detected from `window.location.hostname`
- Demo mode stored in `localStorage.ewap_demo_mode`

13. Development Setup

Prerequisites

- Node.js (LTS version recommended)
- npm or yarn
- MongoDB (local or Atlas URI)

Backend Setup

```
cd backend
npm install
# Create .env with required variables
npm run dev # Starts server on port 3000 with nodemon
```

Frontend Setup

```
cd frontend
npm install
npm run dev # Starts Vite dev server on port 5173
```

Database Utilities

```
# Seed initial data (Roles, Super Admin)
cd backend && node seed.js

# List all users in database
node list_users.js

# Reset Super Admin password
node reset_superuser.js

# Clear entire database (CAUTION)
node clear_db.js
```

14. Deployment Notes

Current Status

- Development environment configured and functional
- Frontend: `http://localhost:5173`
- Backend: `http://localhost:3000`
- Database: MongoDB Atlas (with local fallback)

Production Considerations

- Set `NODE_ENV=production` to enable rate limiting
- Update CORS origins to production domain
- Use strong JWT secrets (min 32 chars)
- Configure proper MongoDB Atlas IP allowlist
- Set up process manager (PM2) for Node.js
- Serve frontend via CDN or static hosting (Vercel, Netlify)
- Backend deployment: Railway, Render, or AWS/GCP

15. Known Issues & Technical Debt

Issue	Severity	Description
<code>AdminDashboard.jsx</code> size	Medium	272KB single file — should be refactored into sub-components
Designation null crash	High	Calculating stats crashes if <code>employee.designation</code> is null
Finance dashboard crash	High	Older payroll records missing <code>salaryHold</code> object cause <code>TypeError</code>
No automated tests	High	Zero unit or integration tests — all testing is manual
Recruitment frontend	Medium	Backend exists but UI is incomplete
Attendance UI	Medium	Backend exists, frontend not fully integrated
No PDF export	Low	Payslip/report PDF generation not implemented
Real-time notifications	Low	Schema exists, full implementation pending
No mobile responsiveness	Medium	Optimized for desktop only
Hardcoded demo data	Low	<code>mockDataEngine.js</code> data is static, not dynamically generated